

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 16
Binary Search Trees

Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline

- Binary Search Trees
  - Definition
  - Tree search
  - Tree sort
  - Insertion
  - Removal

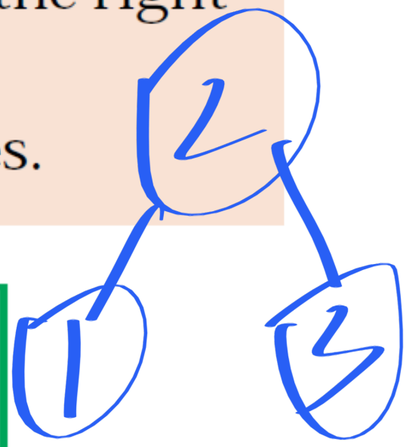
Binary Search Trees

- How to search in a list with frequent insertions and deletions?
  - Linked lists: sequential search is inefficient, though insertions and deletions are easy.
  - Ordered contiguous lists: binary search is efficient, but insertions and deletions are not.
  - Can we somehow keep the advantage s of linked storage and obtain the speed of binary search?
- The answer is YES: Store the list of nodes as a binary tree with the structure of the comparison tree of binary search.

Binary Search Trees

A ***binary search tree*** is a binary tree that is either empty or in which every node has a key (within its data entry) and satisfies the following conditions:

1. The key of the root (if it exists) is greater than the key in any node in the left subtree of the root.
2. The key of the root (if it exists) is less than the key in any node in the right subtree of the root.
3. The left and right subtrees of the root are again binary search trees.



We always require:

No two entries in a binary search tree may have equal keys.

Binary Search Trees

```
template <class Record>
class Search_tree:  public Binary_tree<Record> {
public:
   Error_code insert(const Record &new_data);
   Error_code remove(const Record &old_data);
   Error_code tree_search(Record &target) const;
private:                              //    Add auxiliary function prototypes here.
};
```

```
Error_code Search_tree<Record> ::
            tree_search(Record &target) const;
```

Post: If there is an entry in the tree whose key matches that in target, the parameter target is replaced by the corresponding record from the tree and a code of success is returned. Otherwise a code of not_present is returned.

Binary Search Trees

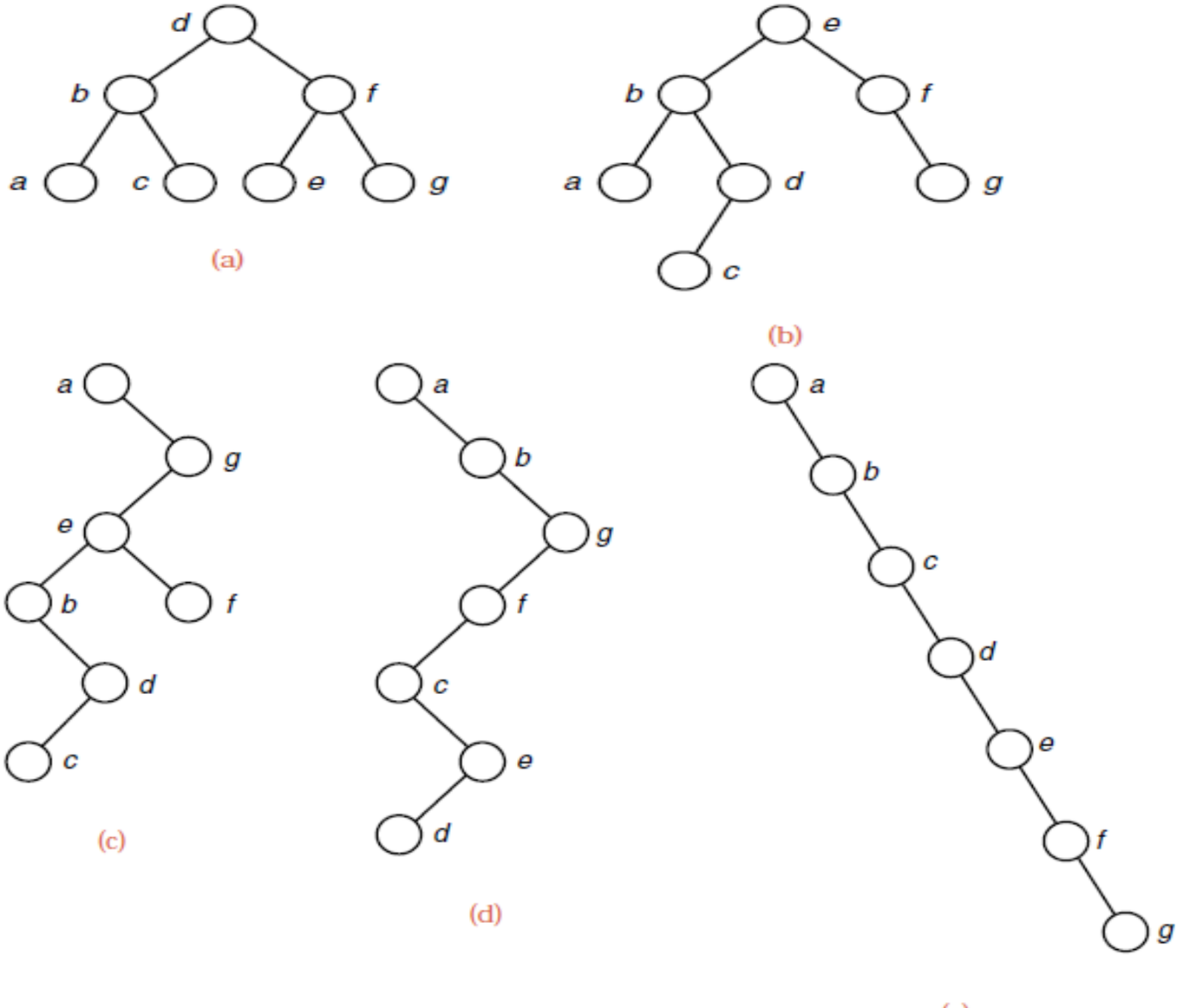
- This method will often be called with a parameter target that contains only a key value. The method will fill target with the complete data belonging to any corresponding Record in the tree.
- To search for the target, we first compare it with the entry at the root of the tree. If their keys match, then we are finished. Otherwise, we go to the left subtree or right subtree as appropriate and repeat the search in that subtree.
- We program this process by calling an auxiliary recursive function.
- The process terminates when it either finds the target or hits an empty subtree.

Recursive and Nonrecursive Auxiliary Functions

```
template <class Record>
Binary_node<Record> *Search_tree<Record> :: search_for_node(
   Binary_node<Record>* sub_root, const Record &target) const
{
   if (sub_root == NULL || sub_root->data == target)
      return sub_root;
   else if (sub_root->data < target)
      return search_for_node(sub_root->right, target);
   else return search_for_node(sub_root->left, target);
}
```

```
template <class Record>
Binary_node<Record> *Search_tree<Record> :: search_for_node(
  Binary_node<Record> *sub_root, const Record &target) const
{
   while (sub_root != NULL && sub_root->data != target)
      if (sub_root->data < target) sub_root = sub_root->right;
      else sub_root = sub_root->left;
   return sub_root;
}
```

Several Binary Search Trees with the Same Keys



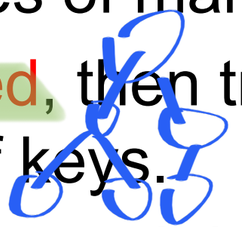
Tree Search

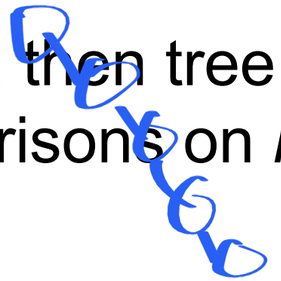
- To search for the target, we first compare it with the entry at the root of the tree. If their keys match, then we are finished. Otherwise, we go to the left subtree or right subtree as appropriate and repeat the search in that subtree.
- We program this process by calling an auxiliary recursive function.
- The process terminates when it either finds the target or hits an empty subtree.

```
template <class Record>
Binary_node<Record> *Search_tree<Record> :: search_for_node(
   Binary_node<Record>* sub_root, const Record &target) const
{
   if (sub_root ==  NULL || sub_root->data ==  target)
      return sub_root;
   else if (sub_root->data < target)
      return search_for_node(sub_root->right, target);
   else return search_for_node(sub_root->left, target);
}
```

Analysis of Tree Search

- The same keys may be built into binary search trees of many different shapes.
- If a binary search tree is nearly completely balanced, then tree search on a tree with *n* vertices will also do *O(log n)* comparisons of keys.
- If the tree degenerates into a long chain, then tree search becomes the same as sequential search, doing *O(n)* comparisons on *n* vertices. This is the worst case for tree search.
- The number of vertices between the root and the target, inclusive, is the number of comparisons that must be done to find the target.
- In practice, if the keys are built into a binary search tree in random order, tree search usually performs almost as well as binary search.

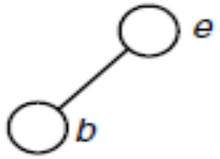




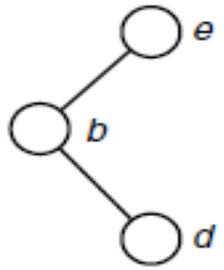
Insertion into a Binary Search Tree



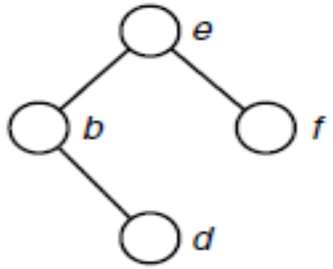
(a) Insert *e*



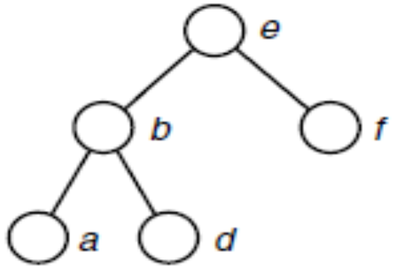
(b) Insert *b*



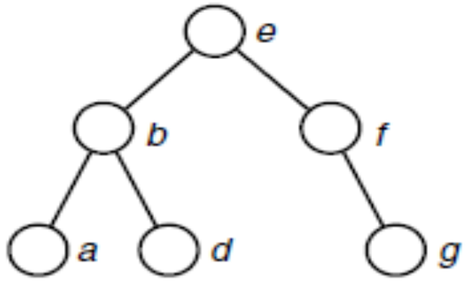
(c) Insert *d*



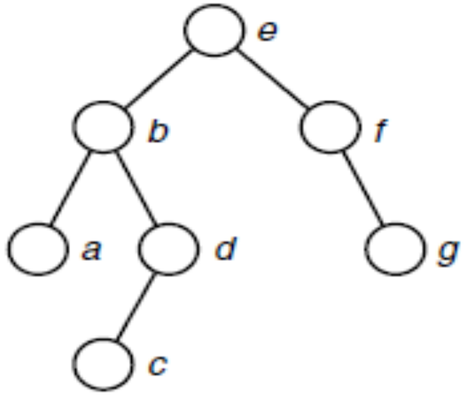
(d) Insert *f*



(e) Insert *a*



(f) Insert *g*



(g) Insert *c*

Method for Insertion

```
template <class Record>
Error_code Search_tree<Record> :: insert(const Record &new_data)
{
  return search_and_insert(root, new_data);
}


template <class Record>
Error_code Search_tree<Record> :: search_and_insert(
            Binary_node<Record> * &sub_root, const Record &new_data)
{
  if (sub_root == NULL) {
    sub_root = new Binary_node<Record>(new_data);
    return success;
  }
  else if (new_data < sub_root->data)
    return search_and_insert(sub_root->left, new_data);
  else if (new_data > sub_root->data)
    return search_and_insert(sub_root->right, new_data);
  else return duplicate_error;
}
```

Performance of Random Binary Search Trees

- Problem: If we assume that the keys have been inserted into a binary search tree in random order, then, on average, how many comparisons are needed?

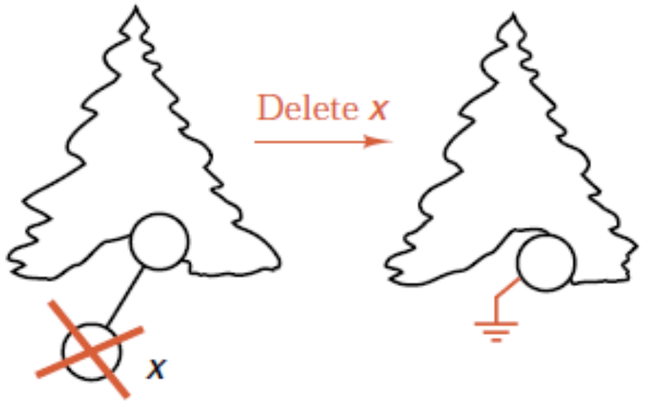
THEOREM 10.3 The average number of nodes visited during a search of the average binary search tree with n nodes is approximately

$$2 \ln n = (2 \ln 2)(\lg n) \approx 1.39 \lg n,$$

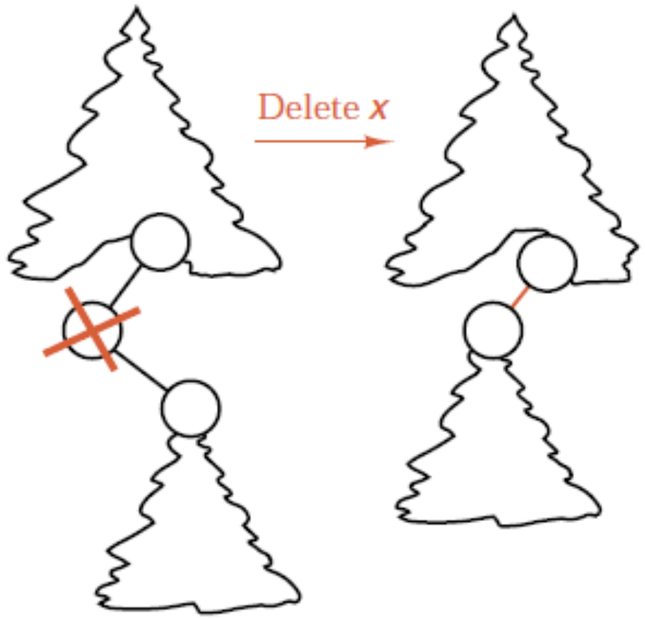
and the number of key comparisons is approximately

$$4 \ln n = (4 \ln 2)(\lg n) \approx 2.77 \lg n.$$

Removal from a Binary Search Tree

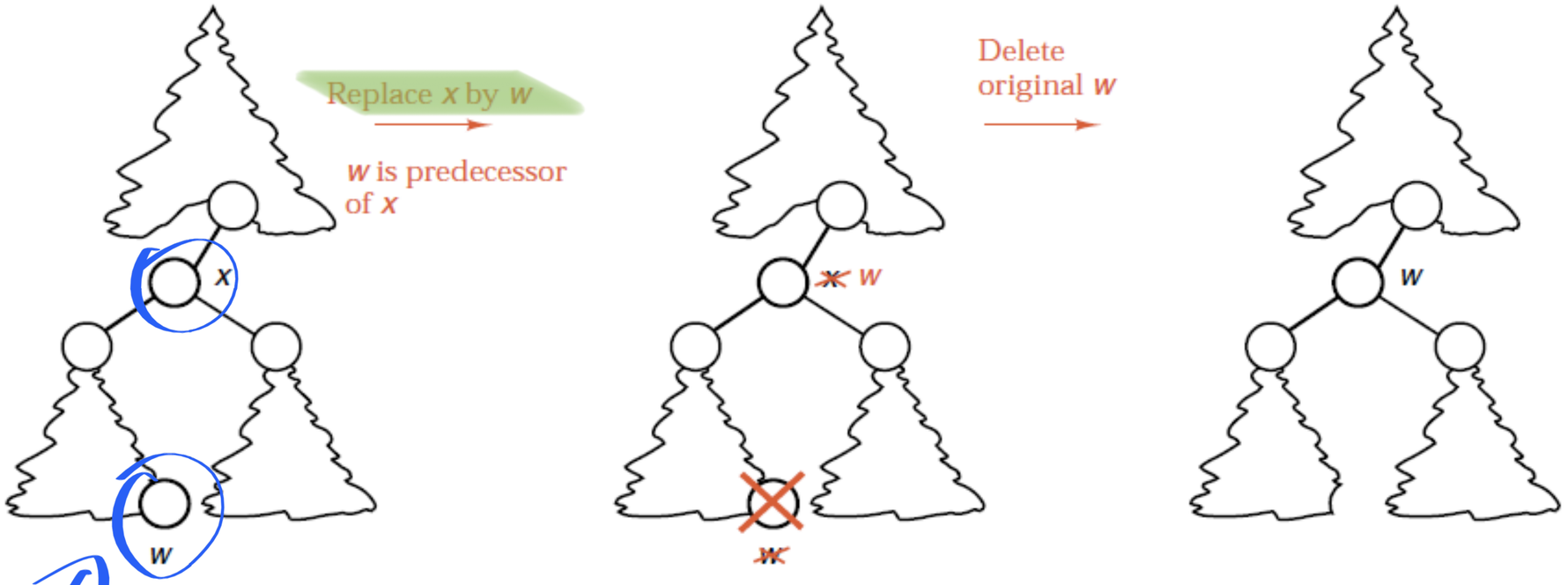


Case: deletion of a leaf



Case: one subtree empty

Removal from a Binary Search Tree



A node's in-order predecessor is the right-most child of its left sub-tree

Removal Method

```
template <class Record>
Error_code Search_tree<Record> :: remove(const Record &target)
/* Post:  If a Record with a key matching that of target belongs to the Search_tree a
          code of success is returned and the corresponding node is removed from the
          tree. Otherwise, a code of not_present is returned.
   Uses: Function search_and_destroy  */

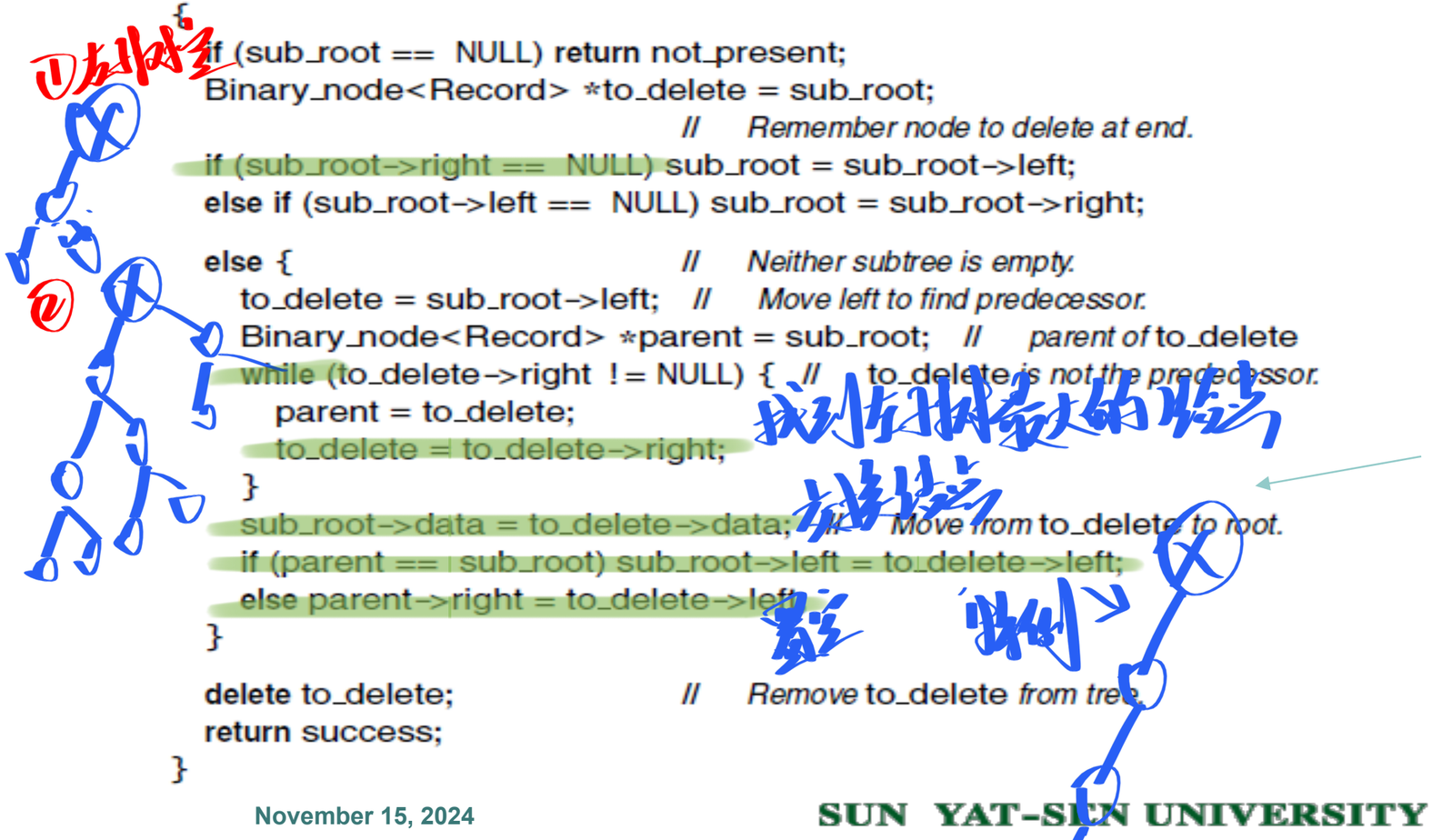
{
  return search_and_destroy(root, target);
}

template <class Record>
Error_code Search_tree<Record> :: search_and_destroy(
              Binary_node<Record>* &sub_root, const Record &target)
/* Pre:   sub_root is either NULL or points to a subtree of the Search_tree.
   Post:  If the key of target is not in the subtree, a code of not_present is returned.
          Otherwise, a code of success is returned and the subtree node containing
          target has been removed in such a way that the properties of a binary search
          tree have been preserved.
   Uses: Functions search_and_destroy recursively and remove_root */

{
  if (sub_root == NULL || sub_root->data == target)
    return remove_root(sub_root);
  else if (target < sub_root->data)
    return search_and_destroy(sub_root->left, target);
  else
    return search_and_destroy(sub_root->right, target);
}
```

Auxiliary Function to Remove One Node

```
template <class Record>
Error_code Search_tree<Record> :: remove_root(Binary_node<Record>
                                              * &sub_root)
/* Pre:  sub_root is either NULL or points to a subtree of the Search_tree.
   Post: If sub_root is NULL, a code of not_present is returned. Otherwise, the root of
         the subtree is removed in such a way that the properties of a binary search tree
         are preserved. The parameter sub_root is reset as the root of the modified
         subtree, and success is returned. */
{
  if (sub_root == NULL) return not_present;
  Binary_node<Record> *to_delete = sub_root;
                                  //  Remember node to delete at end.
  if (sub_root->right == NULL) sub_root = sub_root->left;
  else if (sub_root->left == NULL) sub_root = sub_root->right;
  else {                          //  Neither subtree is empty.
    to_delete = sub_root->left;   //  Move left to find predecessor.
    Binary_node<Record> *parent = sub_root;   //  parent of to_delete
    while (to_delete->right != NULL) {  //  to_delete is not the predecessor.
      parent = to_delete;
      to_delete = to_delete->right;
    }
    sub_root->data = to_delete->data;   //  Move from to_delete to root.
    if (parent == sub_root) sub_root->left = to_delete->left;
    else parent->right = to_delete->left;
  }
  delete to_delete;               //  Remove to_delete from tree.
  return success;
}
```



Note: You should trace through this function to check that all pointers are updated properly, especially in the cases when neither subtree is empty. We must carefully distinguish between the case where the left child of the root is its predecessor and the case where it is necessary to move right to find the predecessor. Note the steps needed in this final case to make the loop stop at a node with an empty right subtree, but not to end at the empty subtree itself.

Exercise

- Draw the binary search tree according to the following operations.

  Insert 3

  Insert 6

  Insert 7

  Insert 2

  Insert 4

  Insert 5

  Delete 6

  Delete 3

Thank you!

